



โครงการ

Object-oriented Programming

Race to Parksoi

จัดทำโดย

6804062612153 นาย เมธาสิทธิ์ วิวัฒน์วรพงศ์

เสนอ

ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เกี่ยวกับโครงการ

ชื่อโปรเจค: Race to Parksoi

นำเสนอโดย: นาย เมธาสิทธิ์ วิวัฒน์วรพงศ์

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

Source Code: <https://github.com/s6804062612153/JavaGameProject/tree/main>

บทที่ 1 ที่มาและความสำคัญของโครงการ

โครงการนี้จัดขึ้นเพื่อวัดผลความสามารถในการเรียนวิชา Object Oriented Programming โดยการนำเรื่องที่เรียนมาสร้างเป็นชิ้นงานในรูปแบบของเกม โดยใช้แนวความคิดการเขียนโปรแกรมแบบเชิงวัตถุ และยังช่วยให้ผู้จัดทำเรียนรู้อุปกรณ์และเครื่องมือ ผู้จัดทำได้สร้างเกมนี้ขึ้นมา

ประเภทของโครงการ

เกม (Game)

ประเภทของโครงการ

1. เพื่อความบันเทิง
2. ฝึกการตอบสนองและการตัดสินใจที่รวดเร็ว
3. ช่วยให้มีสมาธิกับเกมส์

ขอบเขตโครงการ

1. ความต้องการของระบบ
 - ระบบการดำเนินเกมและการเลือกด่าน
 - ระบบควบคุมและการตอบสนอง
 - ระบบอุปสรรคและไอเทม
 - ระบบคะแนน เงื่อนไขการแพ้-ชนะ
 - การบริหารจัดการหน่วยความจำ

2. ตารางเวลาการดำเนินโครงการ (Project Schedule)

ลำดับ	รายการ	25-31 ก.ย.	2-6 ต.ค.	7-14 ต.ค.	15-22 ต.ค.	22-29 ต.ค.	30-31 ต.ค.
1.	หารูปตัวละครและทำ กราฟิกต่างๆ						
2.	ศึกษาเอกสารและข้อมูล ที่เกี่ยวข้อง						
3.	ลงมือเขียนโปรแกรม						
4.	จัดทำเอกสาร						
5.	ตรวจสอบและแก้ไข ข้อผิดพลาด						

บทที่ 2 การพัฒนา

รายละเอียดเกม

เกมส์ Race to Parksoi สวมบทบาทเป็นนักแข่งรถที่ต้องรีบขับรถไปซื้อน้ำปลาที่ปากซอย ตามที่แม่วาน โดยที่จะต้องรีบไปซื้อและรีบกลับมากดแรงค์เกมให้ทันก่อนที่ แรงค์จะรีภายในวันนั้น ทำให้เขาต้องชิงให้เร็วกว่านรก โดยที่ไม่โดนตำรวจจับ และ ไม่เกิดอุบัติเหตุไปเสียก่อน

วิธีการเล่น

ใช้ W A S D หรือปุ่มลูกศร ในการ ควบคุมทิศทางของรถที่เล่นไปด้วยความเร็วสูงเพื่อขับไปหน้าปากซอย

ตัวละคร

รถของผู้เล่น	รถของผู้ใช้ถนนคนอื่น	รถตำรวจ
		
เหรียญ	หลุม	
		

ฉาก

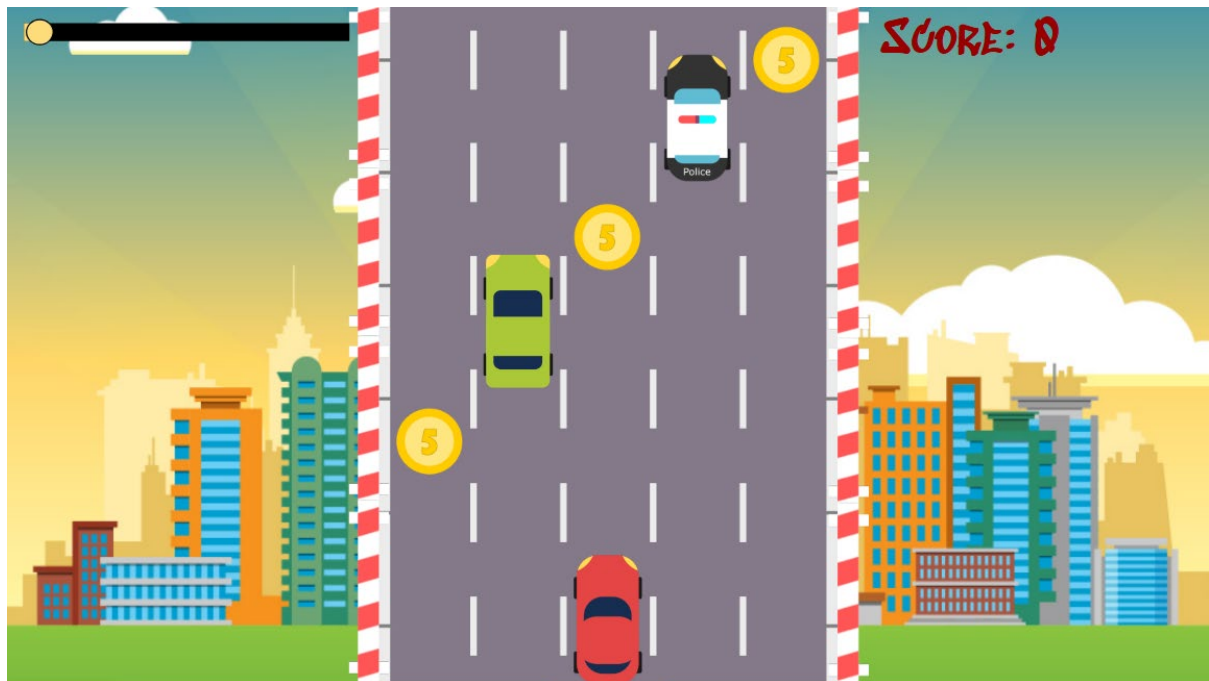
หน้าแรก



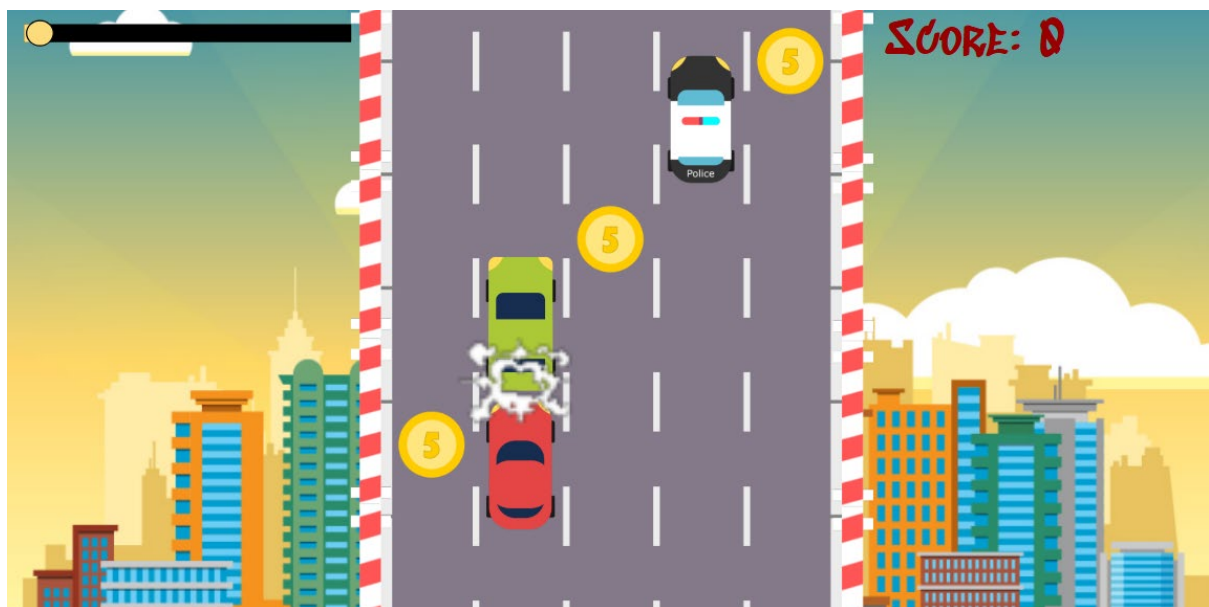
เลือกด่านเพื่อเริ่มเกม



เมื่อเริ่มเกม

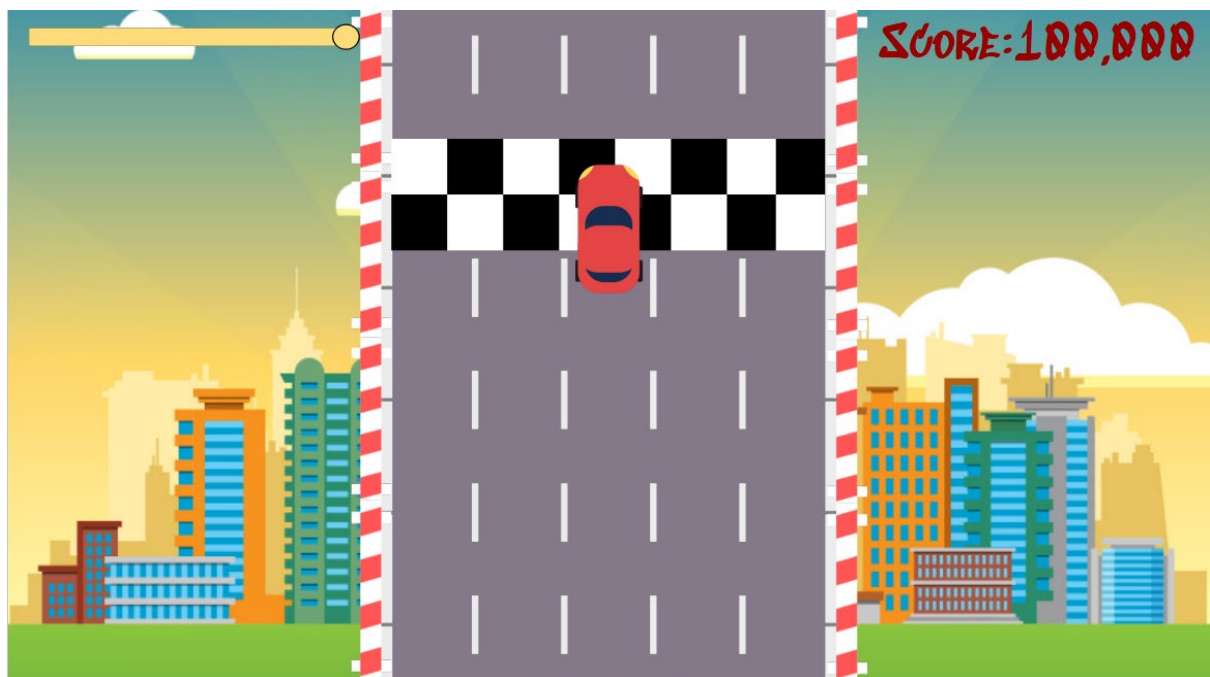


เมื่อเราเกิดชนระหว่างทางจะถือว่าแพ้ทันที



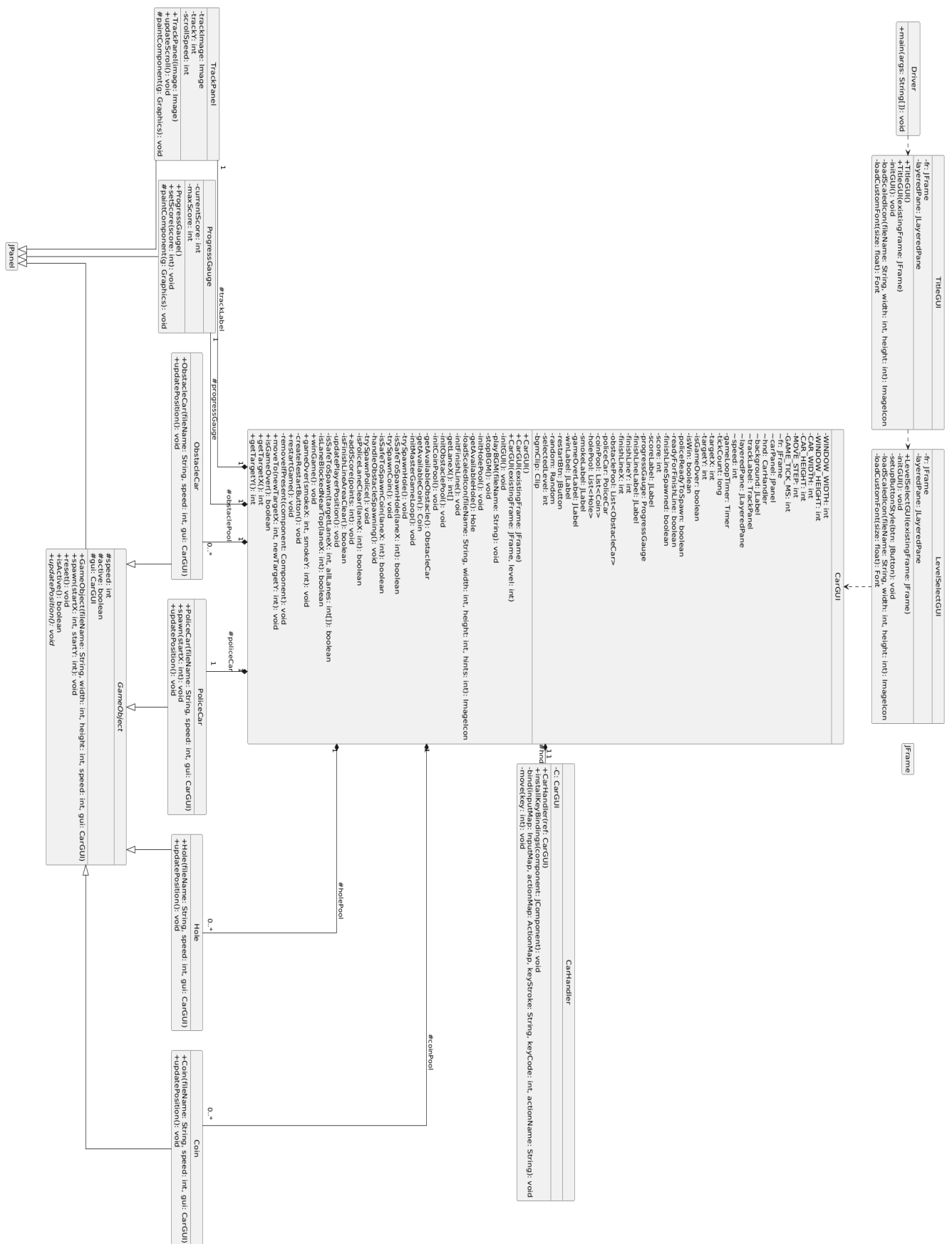


เมื่อเราขับผ่านเส้นชัยได้สำเร็จ จะถือว่าเราชนะ





แผนภาพ Class Diagram



คลาสหลักมี 12 คลาส

สรุปคลาสทั้งหมดของระบบเกมตามรูปแบบที่กำหนด มีรายละเอียดดังนี้:

1. Driver :

○ เมธอด

- main(String[] args) เป็น Method จุดเริ่มต้นการทำงานของโปรแกรม ทำหน้าที่เรียกเปิดหน้าจอเมนูหลัก (TitleGUI) ผ่าน Event Dispatch Thread ของ Swing

2. TitleGUI :

○ ข้อมูลที่เก็บ

- fr (JFrame) หน้าต่างหลักของโปรแกรม
- layeredPane (JLayeredPane) เลเยอร์สำหรับจัดวางรูปภาพพื้นหลัง ข้อความ และปุ่มกด

○ เมธอด

- TitleGUI() Default Constructor สำหรับเรียกสร้างหน้าจอเมนูหลักแบบไม่มีพารามิเตอร์
- TitleGUI(existingFrame: JFrame) Constructor สำหรับสร้างหน้าจอเมนูหลักสามารถรับ Frame เดิมมาใช้งานต่อได้
- initGUI() สร้างและจัดวางองค์ประกอบ UI เช่น พื้นหลัง ข้อความชื่อเกม และปุ่ม "Start" เปลี่ยนเส้นทางไปยัง LevelSelectGUI
- loadScaledIcon(fileName: String, width: int, height: int) โหลดและปรับขนาดรูปภาพไอคอนตามขนาดที่กำหนด
- loadCustomFont(size: float) โหลดฟอนต์ภายนอกมาใช้ในเกม

3. LevelSelectGUI :

○ ข้อมูลที่เก็บ

- fr (JFrame) หน้าต่างหลักของโปรแกรม
- layeredPane (JLayeredPane) เลเยอร์สำหรับจัดวางปุ่มเลือกด่าน

○ เมธอด

- LevelSelectGUI(existingFrame: JFrame) Constructor สำหรับสร้างหน้าเลือกด่าน
- initGUI() สร้างปุ่มเลือกด่าน Level 1 และ Level 2 เมื่อผู้เล่นเลือกด่านใดจะสลับไปยังคลาส CarGUI พร้อมส่งค่าระดับด่านนั้นไป

- `setupButtonStyle(btn: JButton)` กำหนดรูปแบบปุ่มกดให้มีความโปร่งใสและแสดงผลเฉพาะรูปภาพ
- `loadScaledIcon(fileName: String, width: int, height: int)` โหลดและปรับขนาดรูปภาพไอคอน
- `loadCustomFont(size: float)` โหลดฟอนต์ภายนอกมาใช้ในเกม

4. CarGUI :

○ ข้อมูลที่เก็บ

- `WINDOW_WIDTH, WINDOW_HEIGHT (int)` ค่าคงที่กำหนดขนาดความกว้างและความสูงของหน้าต่างเกม
- `CAR_WIDTH, CAR_HEIGHT (int)` ค่าคงที่กำหนดขนาดของรถผู้เล่น
- `MOVE_STEP (int)` ค่าคงที่กำหนดระยะการเคลื่อนที่ของรถ
- `GAME_TICK_MS (int)` ค่าคงที่กำหนดเวลาการทำงานของเกมลูป
- `fr (JFrame)` หน้าต่างหลักของเกม
- `carPanel (JPanel)` พาเนลสำหรับแสดงรูปภาพรถของผู้เล่น
- `hnd (CarHandler)` ตัวจัดการการควบคุมการเคลื่อนที่ด้วยคีย์บอร์ด
- `background (JLabel)` เลเบลสำหรับแสดงภาพพื้นหลังเกม
- `trackLabel (TrackPanel)` พาเนลสำหรับแสดงและจัดการสนามแข่งรถ
- `layeredPane (JLayeredPane)` เลเยอร์สำหรับจัดวางองค์ประกอบ UI และวัตถุต่าง ๆ ในเกม
- `speed (int)` ความเร็วในการเคลื่อนที่ของรถผู้เล่น
- `gameLoopTimer (Timer)` ไทม์เมอร์สำหรับวนลูปอัปเดตสถานะการทำงานของเกม
- `tickCount (long)` ตัวนับจำนวนรอบของเกมลูป
- `targetX, targetY (int)` ตำแหน่งเป้าหมายแกน X และ Y ที่รถผู้เล่นกำลังเคลื่อนที่ไป
- `isGameOver, isWin (boolean)` ตัวแปรระบุสถานะจบเกม (แพ้) และการชนะเกม
- `policeReadyToSpawn, readyForFinishLine, finishLineSpawned (boolean)` ตัวแปรควบคุมสถานะการปล่อยรถตำรวจและเส้นชัย
- `score (int)` คะแนนปัจจุบันของผู้เล่น
- `scoreLabel (JLabel)` เลเบลสำหรับแสดงข้อความคะแนน

- progressGauge (ProgressGauge) เกจแสดงความคืบหน้าของคะแนน
- finishLineLabel (JLabel) เลเบลสำหรับแสดงภาพเส้นชัย
- finishLineY, finishLineX (int) พิกัดตำแหน่งแกน Y และ X ของเส้นชัย
- obstaclePool (List) รายการเก็บวัตถุรถกีดขวาง
- policeCar (PoliceCar) วัตถุรถตำรวจ
- coinPool (List) รายการเก็บวัตถุเหรียญ
- holePool (List) รายการเก็บวัตถุหลุม
- smokeLabel, gameOverLabel, winLabel (JLabel) เลเบลสำหรับแสดงภาพระเบิด ป้าย Game Over และป้าย Win
- restartBtn (JButton) ปุ่มสำหรับเริ่มเกมใหม่หรือกลับไปยังหน้าเมนูหลัก
- random (Random) ตัวสุ่มสำหรับสุ่มการเกิดของวัตถุ
- selectedLevel (int) ระดับด่านที่เลือกเล่น
- bgmClip (Clip) ตัวจัดการเล่นไฟล์เสียงเพลงประกอบเกม

○ เมธอด

- CarGUI() Constructor เริ่มต้นเกมในด่านที่ 1
- CarGUI(existingFrame: JFrame) Constructor เริ่มต้นเกมในด่านที่ 1 โดยใช้ Frame เดิม
- CarGUI(existingFrame: JFrame, level: int) Constructor หลักสำหรับสร้างหน้าจอเล่นเกม กำหนดด่านและความเร็วของเกม
- initGUI() สร้างและจัดวางองค์ประกอบ UI ทั้งหมด เริ่มเล่นเพลงประกอบ และเริ่มต้นเกมลูก
- playBGM(fileName: String) โหลดและเปิดเล่นเพลงประกอบแบบวนลูปต่อเนื่อง
- stopBGM() หยุดการเล่นเพลงประกอบ
- initHolePool() สร้างวัตถุหลุมเข้าสู่พูลตามจำนวนที่กำหนด
- getAvailableHole() ค้นหาและดึงหลุมที่ยังไม่ได้ใช้งานจากพูล
- loadScaledIcon(fileName: String, width: int, height: int, hints: int) โหลดและปรับขนาดรูปภาพไอคอน
- initFinishLine() สร้างและกำหนดพิกัดตำแหน่งเริ่มต้นของเส้นชัย
- getLaneX() คำนวณพิกัดแกน X ของแต่ละเลนการวิ่ง
- initObstaclePool() สร้างวัตถุรถกีดขวางเข้าสู่พูล

- `initCoinPool()` สร้างวัตถุเหรียญเข้าสู่พูล
- `getAvailableObstacle()` ดึงรถกีดขวางที่ยังไม่ได้ใช้งานจากพูล
- `getAvailableCoin()` ดึงเหรียญที่ยังไม่ได้ใช้งานจากพูล
- `initMasterGameLoop()` เริ่มต้นไทม์เมอร์วนลูปอัปเดตการเคลื่อนไหว การเคลื่อนที่ ตรวจสอบจับการชน เพิ่มคะแนน และจัดการการเกิดของวัตถุต่าง ๆ
- `trySpawnHole()` สุ่มสร้างหลุมลงในเลนที่ปลอดภัยสำหรับด้าน 2
- `isSafeToSpawnHole(laneX: int)` ตรวจสอบว่าเลนที่กำหนดปลอดภัยสำหรับการสร้างหลุมหรือไม่
- `trySpawnCoin()` สุ่มสร้างเหรียญลงในเลนที่ปลอดภัย
- `isSafeToSpawnCoin(laneX: int)` ตรวจสอบว่าเลนที่กำหนดปลอดภัยสำหรับการสร้างเหรียญหรือไม่
- `handleObstacleSpawning()` จัดการสุ่มสร้างรถกีดขวางลงในเลนที่ปลอดภัย
- `trySpawnPolice()` สุ่มสร้างรถตำรวจในเลนที่ว่าง
- `isPoliceLaneClear(laneX: int)` ตรวจสอบว่าเลนว่างจากการกีดขวางของรถคันอื่นสำหรับรถตำรวจหรือไม่
- `addScore(points: int)` เพิ่มคะแนนผู้เล่น อัปเดตเกจ และตรวจสอบเงื่อนไขปล่อยรถตำรวจหรือเส้นชัย
- `isFinishLineAreaClear()` ตรวจสอบว่าพื้นที่ด้านบนสนามแข่งว่างเพื่อแสดงเส้นชัยหรือไม่
- `updatePlayerPosition()` อัปเดตการเคลื่อนที่ของรถผู้เล่นไปยังตำแหน่งเป้าหมายแบบนุ่มนวล
- `isSafeToSpawn(targetLaneX: int, allLanes: int[])` ตรวจสอบความปลอดภัยของเลนก่อนสร้างรถกีดขวาง
- `isLaneBlockedNearTop(laneX: int)` ตรวจสอบว่าบริเวณส่วนบนของเลนถูกกีดขวางอยู่หรือไม่
- `winGame()` จัดการสถานะเมื่อผู้เล่นชนะเกม หยุดไทม์เมอร์ แสดงภาพชนะ และสร้างปุ่มกลับหน้าหลัก
- `gameOver(smokeX: int, smokeY: int)` จัดการสถานะเมื่อผู้เล่นแพ้ หยุดไทม์เมอร์ แสดงเอฟเฟกต์ระเบิด ป้าย GameOver และปุ่มเริ่มใหม่
- `createRestartButton()` สร้างและจัดวางปุ่มกดเริ่มเกมใหม่

- restartGame() รีเซ็ตสถานะเกม คะแนน และตำแหน่งของวัตถุทั้งหมดเพื่อเริ่มต้นเล่นเกมใหม่
- removeIfPresent(component: Component) ลบ UI คอมโพเนนต์ออกจากเลย์เออร์
- moveTo(newTargetX: int, newTargetY: int) กำหนดพิกัดเป้าหมายใหม่ให้รถผู้เล่นเคลื่อนที่ไป
- isGameOver() ส่งคืนสถานะว่าเกมจบแล้วหรือชนะแล้วหรือไม่
- getTargetX() ส่งคืนพิกัดเป้าหมายแกน X ของรถผู้เล่น
- getTargetY() ส่งคืนพิกัดเป้าหมายแกน Y ของรถผู้เล่น

5. TrackPanel :

○ ข้อมูลที่เก็บ

- trackImage (Image) รูปภาพสนามแข่ง
- trackY (int) พิกัดแกน Y สำหรับคำนวณการเลื่อนของพื้นหลัง
- scrollSpeed (int) ความเร็วในการเลื่อนสนาม

○ เมธอด

- TrackPanel(image: Image) Constructor รับภาพสนามแข่ง
- updateScroll() อัปเดตพิกัด Y ให้สนามเลื่อนลงอย่างต่อเนื่อง
- paintComponent(g: Graphics) วาดรูปภาพสนามแข่งแบบต่อกันเพื่อสร้างเอฟเฟกต์เลื่อนวนลูป

6. ProgressGauge :

○ ข้อมูลที่เก็บ

- currentScore (int) คะแนนปัจจุบัน
- maxScore (int) คะแนนสูงสุดของเกม (100,000)

○ เมธอด

- ProgressGauge() Constructor กำหนดขนาดและแบบโปรงใส
- setScore(score: int) อัปเดตค่าคะแนนปัจจุบันและสั่งวาดเกมใหม่
- paintComponent(g: Graphics) วาดหลอดเกจแสดงความคืบหน้าของคะแนน

7. CarHandler :

○ ข้อมูลที่เก็บ

- C (CarGUI) การอ้างอิงไปยังคลาส CarGUI

- เมธอด

- CarHandler(ref: CarGUI) Constructor รับการอ้างอิงของ CarGUI
- installKeyBindings(component: JComponent) ติดตั้งการเชื่อมโยงคีย์บอร์ด (Key Bindings) สำหรับปุ่ม WASD และปุ่มลูกศร
- bind(inputMap: InputMap, actionMap: ActionMap, keyStroke: String, keyCode: int, actionName: String) ผูกคีย์สโตรกเข้ากับ Action คำสั่งการเคลื่อนที่
- move(key: int) คำนวณพิกัดใหม่ ตรวจสอบขอบเขตสนามแข่ง และกำหนดตำแหน่งเป้าหมายให้รถผู้เล่น

8. GameObject :

- ข้อมูลที่เก็บ

- speed (int) ความเร็วในการเคลื่อนที่ของวัตถุ
- active (boolean) สถานะเปิดใช้งานของวัตถุ
- gui (CarGUI) การอ้างอิงไปยังหน้าต่างเกมหลัก

- เมธอด

- GameObject(fileName: String, width: int, height: int, speed: int, gui: CarGUI) Constructor กำหนดขนาด รูปภาพ ความเร็ว และการอ้างอิง CarGUI
- spawn(startX: int, startY: int) เปิดใช้งานและกำหนดตำแหน่งเริ่มต้นของวัตถุ
- reset() ปิดการใช้งานวัตถุและย้ายตำแหน่งออกไปนอกหน้าจอ
- isActive() ส่งคืนสถานะการทำงานของวัตถุ
- updatePosition() เมธอด abstract สำหรับอัปเดตตำแหน่งของวัตถุ

9. ObstacleCar :

- เมธอด

- ObstacleCar(fileName: String, speed: int, gui: CarGUI) Constructor สร้างวัตถุรถกีดขวางขนาด 110x220
- updatePosition() อัปเดตตำแหน่งเคลื่อนลงมา ตรวจสอบจับการชนกับรถผู้เล่นหากชนจะเรียก gameOver และรีเซ็ตเมื่อเลยขอบจอด้านล่าง

10. PoliceCar :

- เมธอด

- PoliceCar(fileName: String, speed: int, gui: CarGUI) Constructor สร้างวัตถุ รถตำรวจขนาด 110x220
- spawn(startX: int) กำหนดจุดเกิดเริ่มต้นบริเวณด้านล่างของหน้าจอ
- updatePosition() อัปเดตตำแหน่งเคลื่อนขึ้นไปด้านบน ตรวจสอบการชนกับรถผู้เล่น หากชนจะเรียก gameOver และรีเซ็ตเมื่อเลยขอบจอด้านบน

11. Coin :

○ เมธอด

- Coin(fileName: String, speed: int, gui: CarGUI) Constructor สร้างวัตถุ เหรียญขนาด 120x116
- updatePosition() อัปเดตตำแหน่งเคลื่อนลงมา ตรวจสอบการชนกับรถผู้เล่นเพื่อเพิ่มคะแนน 5,000 คะแนนและรีเซ็ตวัตถุ

12. Hole :

○ เมธอด

- Hole(fileName: String, speed: int, gui: CarGUI) Constructor สร้างวัตถุหลุม ขนาด 110x110
- updatePosition() อัปเดตตำแหน่งเคลื่อนลงมา ตรวจสอบการชนกับรถผู้เล่น หากชนจะเรียก gameOver และรีเซ็ตเมื่อเลยขอบจอด้านล่าง

รูปแบบการพัฒนาโครงการ

- ภาษา: Java
- Framework & Core Libraries (GUI):
 - Java Swing (javax.swing): (JFrame, JLayeredPane, JPanel, JButton, JLabel, Timer, InputMap, ActionMap, AbstractAction, KeyStroke, SwingUtilities, SwingConstants) ไลบรารีหลักสำหรับสร้างส่วนต่อประสานผู้ใช้ (GUI), จัดวางเลย์เออร์หน้าจอ (Layered Pane), ระบบปุ่มกด, การควบคุม Key Bindings สำหรับการเคลื่อนที่และควบคุม Game Loop ผ่าน Swing Timer
 - Java AWT (java.awt & java.awt.event): (Graphics, Graphics2D, BasicStroke, Color, Font, Image, Rectangle, RenderingHints, Cursor, BorderLayout, GraphicsEnvironment, ActionEvent, KeyEvent) ไลบรารีสำหรับการวาดกราฟิก 2D, คำนวณขอบเขตวัตถุและการตรวจจับการชน (Collision Detection), จัดการฟอนต์ภายนอก และรองรับเหตุการณ์จากคีย์บอร์ด
 - Java Sound API (javax.sound.sampled): (AudioInputStream, AudioSystem, Clip) ไลบรารีสำหรับจัดการการเล่นไฟล์เสียงเพลงประกอบ (Background Music) แบบวนลูป
 - Java Utilities & File I/O (java.util & java.io): (ArrayList, List, Random, File) ไลบรารีสำหรับการจัดการโครงสร้างข้อมูลแบบ List, สุ่มตำแหน่งการเกิดของวัตถุ และการโหลดไฟล์ฟอนต์/รูปภาพจาก Directory asset/
- ประเภทซอฟต์แวร์: Standalone Desktop Application
- Asset ภาพที่ใช้: มาจาก <https://swapnilrane24.itch.io/2d-traffic-racer-asset>
- เพลงพื้นหลังที่ใช้ในงาน: https://youtu.be/mcAZ2_mNYDI?si=5EgSyL-d1O_CspV-

แนวความคิดเขียนโปรแกรมเชิงวัตถุ

- Constructor

- คอนสตรัคเตอร์ของคลาส GameObject รับพารามิเตอร์ต่างๆ เช่น ชื่อไฟล์ภาพ ขนาด ความเร็ว และอ้างอิงของ CarGUI เพื่อนำมาตั้งค่าแอตทริบิวต์พื้นฐานและการแสดงผลทางกราฟิกของ JPanel เมื่อคลาสถูก เช่น ObstacleCar, PoliceCar, Coin หรือ Hole สืบทอดมาจาก GameObject จึงไม่จำเป็นต้องเขียนโค้ดตั้งค่าขนาดหรือโหลดรูปภาพใหม่ แต่สามารถเรียกใช้คอนสตรัคเตอร์ของคลาสแม่ผ่านคำสั่ง super(...) โดยส่งเพียงพารามิเตอร์ที่จำเป็นเข้าไปได้ทันที นอกจากนี้ คลาส CarGUI และ TitleGUI ยังมีการซ้อนทับคอนสตรัคเตอร์เพื่อรองรับการเรียกใช้งานในหลายสถานการณ์ เช่น การเปิดเกมใหม่ หรือการรับหน้าจอ JFrame เดิมมาใช้งานต่อผ่านการเรียก this(...)

```
public abstract class GameObject extends JPanel {
    protected int speed;
    protected boolean active = false;
    protected CarGUI gui;

    public GameObject(String fileName, int width, int height, int speed, CarGUI gui) {
        this.speed = speed;
        this.gui = gui;
        setLayout(new BorderLayout());
        setSize(width, height);
        setOpaque(false);
        setDoubleBuffered(true);

        Image img = new ImageIcon("asset/" + fileName).getImage().getScaledInstance(width, height, Image.SCALE_SMOOTH);
        JLabel label = new JLabel(new ImageIcon(img));
        label.setOpaque(false);
        add(label, BorderLayout.CENTER);

        reset();
    }
}

public class ObstacleCar extends GameObject {
    public ObstacleCar(String fileName, int speed, CarGUI gui) {
        super(fileName, 110, 220, speed, gui);
    }
}

public class CarGUI {
    public CarGUI() {
        this(null, 1);
    }

    public CarGUI(JFrame existingFrame) {
        this(existingFrame, 1);
    }

    public CarGUI(JFrame existingFrame, int level) {
        this.selectedLevel = level;
        if (existingFrame != null) {
            this.fr = existingFrame;
            this.fr.setTitle("Race to Parksoi - Level " + level);
        }

        if (selectedLevel == 2) {
            speed = MOVE_STEP + 3;
        } else {
            speed = MOVE_STEP;
        }

        hnd = new CarHandler(this);
        initGUI();
    }
}
```

- Encapsulation

- แอตทริบิวต์ของคลาสหลัก เช่น GameObject ถูกกำหนดเป็นระดับ protected เพื่อจำกัดการมองเห็นให้เฉพาะคลาสที่สืบทอดไป ได้แก่ ObstacleCar, PoliceCar, Coin และ Hole สามารถเข้าถึงได้ ในขณะเดียวกัน แอตทริบิวต์ที่วิกฤตใน CarGUI เช่น score, targetX, targetY และ isGameOver ถูกกำหนดเป็น private เพื่อป้องกันไม่ให้คลาสอื่นแก้ไขค่าได้โดยตรง หากคลาสอื่นอย่าง CarHandler ต้องการอ่านค่า จะต้องเรียกใช้ผ่าน getter methods เช่น getTargetX(), getTargetY() และ isGameOver() หรือหากต้องการแก้ไขตำแหน่งและคะแนน ก็ต้องสั่งการผ่านเมธอดควบคุมอย่าง moveTo() และ addScore() ซึ่งมีการตรวจสอบเงื่อนไขความถูกต้องไว้ภายใน

```
public abstract class GameObject extends JPanel {
    protected int speed;
    protected boolean active = false;
    protected CarGUI gui;

    public boolean isActive() {
        return active;
    }
}

public class CarGUI {
    private int score = 0;
    private int targetX;
    private int targetY;
    private boolean isGameOver = false;
    private boolean isWin = false;

    public boolean isGameOver() {
        return isGameOver || isWin;
    }

    public int getTargetX() {
        return targetX;
    }

    public int getTargetY() {
        return targetY;
    }

    public void addScore(int points) {
        if (isGameOver || isWin) return;
        score = Math.min(score + points, 100000);
        scoreLabel.setText("SCORE: " + score);
        progressGauge.setScore(score);
    }

    public void moveTo(int newTargetX, int newTargetY) {
        if (isGameOver || isWin) return;
        targetX = newTargetX;
        targetY = newTargetY;
    }
}
```

- Composition

- ในคลาส CarGUI มีการประกาศแอตทริบิวต์ที่เป็นวัตถุของคลาสอื่น ได้แก่ CarHandler สำหรับจัดการอินพุตคีย์บอร์ด, TrackPanel สำหรับวาดฉากถนนเคลื่อนที่, ProgressGauge สำหรับแสดงแถบพลัง และการเก็บกลุ่มวัตถุของเกม เช่น ObstacleCar, PoliceCar, Coin และ Hole ในทางกลับกัน คลาส CarHandler ก็มีแอตทริบิวต์เป็นวัตถุของคลาส CarGUI (ผ่านตัวแปร C) เพื่อรับหน้าที่คำนวณขอบเขตลานวิ่งและเรียกใช้เมธอดสั่งการเคลื่อนที่ใน CarGUI

```
public class CarGUI {
    CarHandler hnd;
    TrackPanel trackLabel;
    ProgressGauge progressGauge;

    private final List<ObstacleCar> obstaclePool = new ArrayList<>();
    private PoliceCar policeCar;
    private final List<Coin> coinPool = new ArrayList<>();
    private final List<Hole> holePool = new ArrayList<>();

    public CarGUI(JFrame existingFrame, int level) {
        ...
        hnd = new CarHandler(this);
        initGUI();
    }
}

public class CarHandler {
    private final CarGUI C;

    public CarHandler(CarGUI ref) {
        this.C = ref;
    }

    private void move(int key) {
        if (C.isGameOver()) return;
        int nextTargetX = C.getTargetX();
        int nextTargetY = C.getTargetY();
        ...
        C.moveTo(nextTargetX, nextTargetY);
    }
}
```

- Polymorphism

- ในระบบเกม มีการสร้างคลาสอุปสรรคและไอเทมต่างๆ เช่น ObstacleCar, PoliceCar, Coin และ Hole ซึ่งทั้งหมดสืบทอดมาจาก GameObject คลาส PoliceCar ได้ทำการ Override เมธอด spawn() เพื่อเปลี่ยนจุดเกิดวัตถุให้อยู่ด้านล่างสุดของหน้าจอ (1080) ต่างจากวัตถุชนิดอื่น เมื่อลูปลูกของเกมนั้น CarGUI ทำงาน ระบบจะเรียกเมธอด updatePosition() บนวัตถุแต่ละชนิด โดยวัตถุแต่ละชิ้นจะประมวลผลโลจิกเฉพาะตัว เช่น ObstacleCar เลื่อนลงและทำให้เกมจบเมื่อชน, PoliceCar เลื่อนขึ้นจากด้านล่าง, และ Coin เลื่อนลงพร้อมเพิ่มคะแนนให้ผู้เล่น

```
public class PoliceCar extends GameObject {
    public PoliceCar(String fileName, int speed, CarGUI gui) {
        super(fileName, 110, 220, speed, gui);
    }

    public void spawn(int startX) {
        super.spawn(startX, 1080);
    }
}

private void initMasterGameLoop() {
    gameLoopTimer = new Timer(GAME_TICK_MS, e -> {
        if (isGameOver || isWin) return;
        tickCount++;

        trackLabel.updateScroll();
        updatePlayerPosition();

        for (ObstacleCar obs : obstaclePool) {
            obs.updatePosition();
        }
        policeCar.updatePosition();

        for (Coin c : coinPool) {
            c.updatePosition();
        }

        if (selectedLevel == 2) {
            for (Hole h : holePool) {
                h.updatePosition();
            }
        }
        ...
    });
    gameLoopTimer.start();
}
```

- Abstract & Inheritance

- คลาส GameObject สืบทอดมาจาก JPanel และถูกกำหนดเป็น abstract class โดยมี Abstract Method คือ getOutput() หรือในที่นี้คือ updatePosition() เพื่อส่งสัญญาณบังคับให้คลาสที่สืบทอดไปต้องเขียนการทำงานจริง ตัวอย่างเช่น คลาส Coin สืบทอดมาจาก GameObject แล้ว Implement เมธอด updatePosition() โดยจะเลื่อนตำแหน่งเหรียญลงมา ตรวจสอบการชนกับรถผู้เล่น หากชนกันจะส่งคะแนนเพิ่ม 5,000 คะแนนให้แก่ CarGUI และสร้างชีวิตตนเอง ในขณะที่คลาส Hole สืบทอดมาจาก GameObject เช่นกัน แต่ Implement เมธอด updatePosition() ให้คำนวณจุดตัดการชนแล้วส่งคำสั่ง gui.gameOver(...) เพื่อยุติการเล่นเกม

```
public abstract class GameObject extends JPanel {
    protected int speed;
    protected boolean active = false;
    protected CarGUI gui;

    public abstract void updatePosition();
}

public class Coin extends GameObject {
    public Coin(String fileName, int speed, CarGUI gui) {
        super(fileName, 120, 116, speed, gui);
    }

    @Override
    public void updatePosition() {
        if (!active || gui.isGameOver()) return;

        int nextY = getY() + speed;
        setLocation(getX(), nextY);

        Rectangle playerBounds = gui.carPanel.getBounds();
        Rectangle coinBounds = getBounds();

        if (playerBounds.intersects(coinBounds)) {
            gui.addScore(5000);
            reset();
            return;
        }

        if (nextY > 1080) {
            reset();
        }
    }
}
```

```

public class Hole extends GameObject {
    public Hole(String fileName, int speed, CarGUI gui) {
        super(fileName, 110, 110, speed, gui);
    }

    @Override
    public void updatePosition() {
        if (!active || gui.isGameOver()) return;

        int nextY = getY() + speed;
        setLocation(getX(), nextY);

        Rectangle playerBounds = gui.carPanel.getBounds();
        Rectangle holeBounds = getBounds();

        if (playerBounds.intersects(holeBounds)) {
            Rectangle intersection = playerBounds.intersection(holeBounds);
            int centerX = (int) intersection.getCenterX();
            int centerY = (int) intersection.getCenterY();
            gui.gameOver(centerX - 70, centerY - 66);
            return;
        }

        if (nextY > 1080) {
            reset();
        }
    }
}

```

อัลกอริทึมที่สำคัญ

- Master Game Loop Algorithm

- อัลกอริทึมการควบคุมวงรอบหลักของเกม อาศัยการทำงานของ javax.swing.Timer ในการส่งสัญญาณนาฬิกา (Tick) ทุกๆ 16 มิลลิวินาที อัลกอริทึมนี้ทำหน้าที่เป็นตัวกลางในการสั่งการอัปเดตสถานะของทุกเอนทิตีในเกมตามลำดับความสำคัญ ฟังก์ชัน initMasterGameLoop ทำหน้าที่เป็นจุดศูนย์กลางในการประมวลผล logic ทั้งหมดของเกมในทุกๆ เฟรม เริ่มจากการตรวจสอบสถานะเกม (isGameOver หรือ isWin) จากนั้นจะเพิ่มค่า tickCount เพื่อใช้นับรอบเวลา อัลกอริทึมจะสั่งอัปเดตตำแหน่งการเลื่อนของถนน (trackLabel.updateScroll()) อัปเดตพิกัดรถผู้เล่น (updatePlayerPosition()) จากนั้นจึงสั่งอัปเดตตำแหน่งสิ่งกีดขวาง รถตำรวจ เหยี่ยว และหลุม (ตามระดับความยาก) นอกจากนี้ยังจัดการจังหวะการสะสมคะแนนและการเรียกใช้อัลกอริทึมการเกิด (Spawn Logic) ของวัตถุต่างๆ ตามรอบเวลา tickCount ตลอดจนการตรวจสอบเงื่อนไขการเกิดเส้นชัยและการชนะเกม อยู่ที่ CarGUI.java

```

private void initMasterGameLoop() {
    gameLoopTimer = new Timer(GAME_TICK_MS, e -> {
        if (isGameOver || isWin) return;

        tickCount++;

        trackLabel.updateScroll();

        updatePlayerPosition();

        for (ObstacleCar obs : obstaclePool) {
            obs.updatePosition();
        }
        policeCar.updatePosition();

        for (Coin c : coinPool) {
            c.updatePosition();
        }

        if (selectedLevel == 2) {
            for (Hole h : holePool) {
                h.updatePosition();
            }
        }

        if (tickCount % 6 == 0) {
            addScore(54);
        }

        if (tickCount % 75 == 0 && !readyForFinishLine && !finishLineSpawned) {
            handleObstacleSpawning();
        }

        if (tickCount % 90 == 0 && !readyForFinishLine && !finishLineSpawned) {
            trySpawnCoin();
        }

        if (selectedLevel == 2 && tickCount % 120 == 0 && !readyForFinishLine && !finishLineSpawned) {
            trySpawnHole();
        }

        if (policeReadyToSpawn && !policeCar.isActive()) {
            trySpawnPolice();
        }

        if (readyForFinishLine && !finishLineSpawned) {
            if (isFinishLineAreaClear()) {
                finishLineSpawned = true;
                finishLineLabel.setVisible(true);
            }
        }

        if (finishLineSpawned) {
            finishLineY += 3;
            finishLineLabel.setLocation(finishLineX, finishLineY);

            Rectangle carBounds = carPanel.getBounds();
            Rectangle finishBounds = finishLineLabel.getBounds();
            if (carBounds.intersects(finishBounds)) {
                winGame();
            }
        }
    });
    gameLoopTimer.start();
}

```

- Dynamic Score Calculation & Police Event Trigger Algorithm

- อัลกอริทึมสะสมคะแนนแบบไดนามิกและการเปิดทำงานของเหตุการณ์รถตำรวจฉุกเฉิน ทำหน้าที่คำนวณคะแนนผู้เล่น ควบคุมขอบเขตคะแนนไม่ให้เกิน 100,000 คะแนน และตรวจสอบเงื่อนไข เพื่อสปอนรถตำรวจ เมธอด addScore รับค่า points แล้วคำนวณคะแนนใหม่โดยใช้ Math.min เพื่อจำกัดคะแนนสูงสุดไว้ที่ 100,000 จากนั้นปรับปรุงการแสดงผลผ่าน scoreLabel และ progressGauge ส่วนสำคัญคืออัลกอริทึมตรวจจับการข้ามช่วงทุกๆ 10,000 คะแนน โดยเปรียบเทียบการหารจำนวนเต็ม $(score / 10000) > (oldScore / 10000)$ หากมีการข้ามช่วงและรถตำรวจยังไม่อยู่ในสถานะทำงาน ระบบจะกำหนด policeReadyToSpawn = true เพื่อเตรียมปล่อยรถตำรวจในรอบถัดไป รวมถึงเปลี่ยนสถานะ readyForFinishLine = true เมื่อคะแนนถึงแต้มสูงสุด อยู่ที่ CarGUI.java

```
public void addScore(int points) {  
    if (isGameOver || isWin) return;  
  
    int oldScore = score;  
    score = Math.min(score + points, 100000);  
    scoreLabel.setText("SCORE: " + score);  
    progressGauge.setScore(score);  
  
    if (score > 0 && (score / 10000) > (oldScore / 10000)) {  
        if (!policeCar.isActive()) {  
            policeReadyToSpawn = true;  
        }  
    }  
  
    if (score >= 100000 && !readyForFinishLine && !finishLineSpawned) {  
        readyForFinishLine = true;  
    }  
}
```

- Game State Transition & Reset Algorithm

- อัลกอริทึมการเปลี่ยนสถานะและการคืนค่าสถานะเกม (Reset) มีหน้าที่ทำลายวัตถุชั่วคราว
ล้างสถานะในกลุ่มเอนทิตี (Pools) และตั้งค่าพิกัดเริ่มต้นใหม่ทั้งหมดเมื่อผู้เล่นกดเริ่มต้นเกม
ใหม่ เมื่อกด restartGame ปฏิบัติการคืนค่าระบบ โดยลบ UI ส่วนปุ่ม Restart และป้าย
Game Over/Win ออกจาก layeredPane จากนั้นสั่งวนลูปเรียกเมธอด reset() บนเอนทิตี
ทุกตัวใน obstaclePool, coinPool, holePool และ policeCar เพื่อนำวัตถุกลับเข้าสู่
สถานะซ่อนตัวและปิดการทำงาน พิกัดของรถผู้เล่นและเป้าหมายจะถูกรีเซ็ตกลับไปพิกัด
เริ่มต้น (905, 800) พร้อมกับเริ่มเล่นเพลง background และสั่งให้ gameLoopTimer
ทำงานอีกครั้ง อยู่ที่ CarGUI.java

```
public void restartGame() {  
    removeIfPresent(smokeLabel);  
    removeIfPresent(gameOverLabel);  
    removeIfPresent(winLabel);  
    removeIfPresent(restartBtn);  
  
    smokeLabel = null;  
    gameOverLabel = null;  
    winLabel = null;  
    restartBtn = null;  
  
    score = 0;  
    tickCount = 0;  
    scoreLabel.setText("SCORE: 0");  
    progressGauge.setScore(0);  
  
    for (ObstacleCar obs : obstaclePool) {  
        obs.reset();  
    }  
    for (Coin c : coinPool) {  
        c.reset();  
    }  
    if (selectedLevel == 2) {  
        for (Hole h : holePool) {  
            h.reset();  
        }  
    }  
    policeCar.reset();  
  
    readyForFinishLine = false;  
    finishLineSpawned = false;  
    finishLineY = -171;  
    finishLineLabel.setLocation(finishLineX, finishLineY);  
    finishLineLabel.setVisible(false);  
  
    int startX = 905;  
    int startY = 800;  
    carPanel.setLocation(startX, startY);  
    targetX = startX;  
    targetY = startY;  
  
    isGameOver = false;  
    isWin = false;  
    policeReadyToSpawn = false;  
  
    playBGM("bgm.wav");  
    gameLoopTimer.start();  
  
    layeredPane.revalidate();  
    layeredPane.repaint();  
    SwingUtilities.invokeLater(() -> layeredPane.requestFocusInWindow());  
}
```

- Event Dispatch Thread Initialization Algorithm

- อัลกอริทึมการเริ่มต้นระบบการทำงานของหลักของเกม ผ่าน Event Dispatch Thread อธิบายการทำงานและส่วนที่เกี่ยวข้อง: เมธอด main ใช้อัลกอริทึมส่งการสร้างวัตถุ TitleGUI เข้าไปในคิวของ Event Dispatch Thread ผ่าน SwingUtilities.invokeLater เพื่อหลีกเลี่ยงปัญหาการเข้าถึงและแก้ไขส่วนประกอบของ GUI พร้อมกันจากหลายเธรด (Race Conditions) อยู่ที่ Driver.java

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(new TitleGUI());  
}
```

- Key Binding Mapping & Input Dispatch Algorithm

- เมธอด installKeyBindings ทำการลงทะเบียนคำสั่งเมื่อมีการกดปุ่มลงใน InputMap ในขอบเขต WHEN_IN_FOCUSED_WINDOW จากนั้นผูกชื่อแอ็กชันเข้ากับอินสแตนซ์ของ AbstractAction ใน ActionMap ซึ่งเมื่อเกิดเหตุการณ์กดปุ่ม อัลกอริทึมจะส่งต่อรหัสปุ่ม (keyCode) ไปยังเมธอด move เพื่อดำเนินการเปลี่ยนแปลงพิกัดเป้าหมายทันที อยู่ที่ CarHandler.java

```
public void installKeyBindings(JComponent component) {  
    InputMap inputMap = component.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);  
    ActionMap actionMap = component.getActionMap();  
  
    // WASD Key Bindings  
    bind(inputMap, actionMap, "pressed A", KeyEvent.VK_A, "moveLeft");  
    bind(inputMap, actionMap, "pressed D", KeyEvent.VK_D, "moveRight");  
    bind(inputMap, actionMap, "pressed W", KeyEvent.VK_W, "moveUp");  
    bind(inputMap, actionMap, "pressed S", KeyEvent.VK_S, "moveDown");  
  
    // Arrow Key Bindings  
    bind(inputMap, actionMap, "pressed LEFT", KeyEvent.VK_LEFT, "moveLeft");  
    bind(inputMap, actionMap, "pressed RIGHT", KeyEvent.VK_RIGHT, "moveRight");  
    bind(inputMap, actionMap, "pressed UP", KeyEvent.VK_UP, "moveUp");  
    bind(inputMap, actionMap, "pressed DOWN", KeyEvent.VK_DOWN, "moveDown");  
}  
  
private void bind(InputMap inputMap, ActionMap actionMap,  
    String keyStroke, int keyCode, String actionName) {  
    KeyStroke stroke = KeyStroke.getKeyStroke(keyStroke);  
    inputMap.put(stroke, actionName);  
    actionMap.put(actionName, new AbstractAction() {  
        @Override  
        public void actionPerformed(ActionEvent e) {  
            move(keyCode);  
        }  
    });  
}
```

- Grid-Based Lane Boundary Clamping Algorithm

- อัลกอริทึมการคำนวณตำแหน่งเป้าหมายและการจำกัดขอบเขตการเคลื่อนที่ (Boundary Clamping) เพื่อคำนวณพิกัดใหม่ของรถตามความกว้างของเลน พร้อมป้องกันไม่ให้รถหลุดออกนอกขอบถนน เมธอด move คำนวณพิกัดขอบเขตความปลอดภัยของถนน (minLeftX, maxRightX, minY, maxY) จากขนาดและตำแหน่งของ trackLabel และ carPanel เมื่อผู้เล่นสั่งเคลื่อนที่ อัลกอริทึมจะบวกหรือลบระยะ 138 พิกเซลจากพิกัดเป้าหมายเดิม แล้วใช้อัลกอริทึม Math.max และ Math.min ในการจำกัดพิกัดไม่ให้เกินขอบเขตที่กำหนด ก่อนส่งพิกัดเป้าหมายใหม่ไปให้ CarGUI ผ่าน moveTo อยู่ที่ CarHandler.java

```
private void move(int key) {
    if (C.isGameOver()) return;

    int minLeftX = C.trackLabel.getX() + 68;
    int maxRightX = C.trackLabel.getX()
        + C.trackLabel.getWidth()
        - C.carPanel.getWidth()
        - 68;

    int minY = C.trackLabel.getY();
    int maxY = C.trackLabel.getY()
        + C.trackLabel.getHeight()
        - C.carPanel.getHeight();

    int nextTargetX = C.getTargetX();
    int nextTargetY = C.getTargetY();

    if (key == KeyEvent.VK_A || key == KeyEvent.VK_LEFT) {
        nextTargetX = Math.max(minLeftX, nextTargetX - 138);
    } else if (key == KeyEvent.VK_D || key == KeyEvent.VK_RIGHT) {
        nextTargetX = Math.min(maxRightX, nextTargetX + 138);
    } else if (key == KeyEvent.VK_W || key == KeyEvent.VK_UP) {
        nextTargetY = Math.max(minY, nextTargetY - 138);
    } else if (key == KeyEvent.VK_S || key == KeyEvent.VK_DOWN) {
        nextTargetY = Math.min(maxY, nextTargetY + 138);
    }

    C.moveTo(nextTargetX, nextTargetY);
}
```

- Smooth Target-Based Position Interpolation Algorithm

- อัลกอริทึมการสอดแทรกตำแหน่งแบบก้าวเดียว (Linear Step Interpolation) เพื่อปรับพิกัดจริงของรถเข้าหาพิกัดเป้าหมายอย่างราบรื่นในทุกๆ เฟรมการทำงานของ Master Game Loop เมื่อ updatePlayerPosition ดึงพิกัดปัจจุบันเทียบกับ targetX และ targetY หากระยะห่างน้อยกว่าหรือเท่ากับค่า step (ซึ่งกำหนดโดยตัวแปร speed) อัลกอริทึมจะกำหนดให้พิกัดถัดไปเท่ากับพิกัดเป้าหมายทันที แต่หากระยะห่างมากกว่า อัลกอริทึมจะเลื่อนพิกัดไปที่ละ step ทั้งแกน X และ Y ส่งผลให้การเปลี่ยนแปลงของรถมีภาพเคลื่อนไหวที่นุ่มนวล ไม่วาร์ปไปทันที อยู่ที่ CarGUI.java

```
private void updatePlayerPosition() {  
    int currentX = carPanel.getX();  
    int currentY = carPanel.getY();  
    int step = speed;  
  
    int nextX = currentX;  
    if (Math.abs(currentX - targetX) <= step) {  
        nextX = targetX;  
    } else if (currentX < targetX) {  
        nextX = currentX + step;  
    } else {  
        nextX = currentX - step;  
    }  
  
    int nextY = currentY;  
    if (Math.abs(currentY - targetY) <= step) {  
        nextY = targetY;  
    } else if (currentY < targetY) {  
        nextY = currentY + step;  
    } else {  
        nextY = currentY - step;  
    }  
  
    if (nextX != currentX || nextY != currentY) {  
        carPanel.setLocation(nextX, nextY);  
    }  
}
```

- Object Pooling & Lifecycle Management Algorithm

- โครงสร้างอัลกอริทึมการรวมกลุ่มวัตถุ (Object Pooling Pattern) ในคลาสฐาน GameObject ช่วยหลีกเลี่ยงการสร้างและทำลายวัตถุซ้ำๆ คลาส GameObject กำหนดการทำงานพื้นฐานของเอนิตีทั้งหมดในเกม เมธอด spawn จะเปิดใช้งานวัตถุและกำหนดตำแหน่งพิกัดเริ่มต้น ขณะที่เมธอด reset จะปิดการทำงานของวัตถุ (active = false), ซ่อนการแสดงผล (setVisible(false)) และย้ายตำแหน่งออกไปนอกหน้าจอที่พิกัด (-500, -500) เพื่อนำกลับมาหมุนเวียนใช้ใหม่ได้ทันที อยู่ที่ GameObject.java

```
public abstract class GameObject extends JPanel {
    protected int speed;
    protected boolean active = false;
    protected CarGUI gui;

    public GameObject(String fileName, int width, int height, int speed, CarGUI gui) {
        this.speed = speed;
        this.gui = gui;
        setLayout(new BorderLayout());
        setSize(width, height);
        setOpaque(false);
        setDoubleBuffered(true);

        Image img = new ImageIcon("asset/" + fileName).getImage().getScaledInstance(width, height, Image.SCALE_SMOOTH);
        JLabel label = new JLabel(new ImageIcon(img));
        label.setOpaque(false);
        add(label, BorderLayout.CENTER);

        reset();
    }

    public void spawn(int startX, int startY) {
        setLocation(startX, startY);
        active = true;
        setVisible(true);
    }

    public void reset() {
        active = false;
        setVisible(false);
        setLocation(-500, -500);
    }

    public boolean isActive() {
        return active;
    }

    public abstract void updatePosition();
}
```

- Safe Lane Obstacle Spawning & Spatial Avoidance Algorithm

- อัลกอริทึมการตรวจสอบและสุ่มเกิดรถสิ่งกีดขวางเพื่อป้องกันการเกิดสถานะที่ผู้เล่นไม่สามารถหลบได้อธิบายการทำงานและส่วนที่เกี่ยวข้อง: อัลกอริทึมนี้จะประเมินความปลอดภัยของแต่ละเลนก่อนปล่อยรถกีดขวาง ฟังก์ชัน `isSafeToSpawn` มีเงื่อนไขการตรวจสอบ 3 อย่าง ได้แก่

-ระยะห่างแนวตั้งขั้นต่ำ ต้องมีระยะห่างจากรถคันอื่นในเลนเดียวกันอย่างน้อย 320 พิกเซล

-จำนวนเลนที่ถูกปิดกั้น หากการปล่อยรถคันนี้ส่งผลให้มีเลนถูกปิดกั้นพร้อมกันตั้งแต่ 3 เลนขึ้นไปในพื้นที่ด้านบน จะยกเลิกการปล่อยทันทีเพื่อเว้นช่องทางรอด

-การป้องกันมุมอับขอบถนน (Corner Trap Prevention): ตรวจสอบตำแหน่งผู้เล่น หากผู้เล่นชิดขอบซ้ายหรือขวา และเลนข้างเคียงถูกบล็อก จะไม่ปล่อยรถเข้าซ้อนในพื้นที่ปิดทางรอดของผู้เล่น อยู่ที่ `CarGUI.java`

```
private void handleObstacleSpawning() {
    int[] lanesX = getLaneX();
    List<Integer> safeLanes = new ArrayList<>();
    for (int laneX : lanesX) {
        if (isSafeToSpawn(laneX, lanesX)) {
            safeLanes.add(laneX);
        }
    }

    if (!safeLanes.isEmpty()) {
        ObstacleCar availableCar = getAvailableObstacle();
        if (availableCar != null) {
            int randomLane = safeLanes.get(random.nextInt(safeLanes.size()));
            availableCar.spawn(randomLane, -CAR_HEIGHT);
        }
    }
}

private boolean isSafeToSpawn(int targetLaneX, int[] allLanes) {
    int minVerticalGap = 320;

    for (ObstacleCar obs : obstaclePool) {
        if (obs.isActive())
            && obs.getX() == targetLaneX
            && obs.getY() < minVerticalGap {
                return false;
            }
    }

    int blockedLanes = 0;
    for (int laneX : allLanes) {
        if (laneX == targetLaneX) {
            blockedLanes++;
            continue;
        }
        for (ObstacleCar obs : obstaclePool) {
            if (obs.isActive())
                && obs.getX() == laneX
                && obs.getY() >= -CAR_HEIGHT
                && obs.getY() <= 250 {
                    blockedLanes++;
                    break;
                }
        }
    }

    if (blockedLanes >= 3) return false;

    int playerX = carPanel.getX();
    int leftEdgeLane = allLanes[0];
    int rightEdgeLane = allLanes[allLanes.length - 1];

    if (Math.abs(playerX - leftEdgeLane) < 50) {
        if (targetLaneX == allLanes[1] && isLaneBlockedNearTop(leftEdgeLane)) {
            return false;
        }
        if (targetLaneX == leftEdgeLane && isLaneBlockedNearTop(allLanes[1])) {
            return false;
        }
    }

    if (Math.abs(playerX - rightEdgeLane) < 50) {
        if (targetLaneX == allLanes[2] && isLaneBlockedNearTop(rightEdgeLane)) {
            return false;
        }
        if (targetLaneX == rightEdgeLane && isLaneBlockedNearTop(allLanes[2])) {
            return false;
        }
    }

    return true;
}
```

- Safe Lane Coin Spawning Algorithm

- อัลกอริทึมที่สุ่มตำแหน่งเกิดเหรียญสะสมคะแนนบนเลนที่ปลอดภัย ตรวจสอบไม่ให้เกิดทับซ้อนกับรถสิ่งกีดขวางหรือรถตำรวจ ฟังก์ชัน trySpawnCoin ดึงเหรียญที่ว่างจาก coinPool แล้วตรวจสอบทุกเลนด้วย isSafeToSpawnCoin หากเลนใดไม่มีรถสิ่งกีดขวางหรือรถตำรวจแล่นอยู่ในพิกัด Y ช่วง -220 ถึง 350 พิกเซล เลนนั้นจะถูกจัดเป็นเลนปลอดภัย แล้วทำการสุ่มเลนเพื่อวางเหรียญที่พิกัดกึ่งกลางเลน อยู่ที่ CarGUI.java

```
private void trySpawnCoin() {
    Coin availableCoin = getAvailableCoin();
    if (availableCoin == null) return;

    int[] lanesX = getLaneX();
    List<Integer> safeLanes = new ArrayList<>();

    for (int laneX : lanesX) {
        if (isSafeToSpawnCoin(laneX)) {
            safeLanes.add(laneX);
        }
    }

    if (!safeLanes.isEmpty()) {
        int randomLaneX = safeLanes.get(random.nextInt(safeLanes.size()));
        int coinX = randomLaneX + (CAR_WIDTH - 120) / 2;
        availableCoin.spawn(coinX, -116);
    }
}

private boolean isSafeToSpawnCoin(int laneX) {
    for (ObstacleCar obs : obstaclePool) {
        if (obs.isActive() && obs.getX() == laneX && obs.getY() >= -CAR_HEIGHT && obs.getY() <= 350) {
            return false;
        }
    }

    if (policeCar.isActive() && policeCar.getX() == laneX && policeCar.getY() >= -CAR_HEIGHT && policeCar.getY() <= 350) {
        return false;
    }

    return true;
}
```

- Safe Hole Spawning Algorithm

- อัลกอริทึมสุ่มเกิดหลุมตก (สำหรับ Level 2) โดยมีการตรวจสอบการทับซ้อนกับรถPlayer ฟังก์ชัน trySpawnHole สำหรับด่านระดับ 2 จะสุ่มหาเลน่ว่างโดยเช็คความปลอดภัยผ่าน isSafeToSpawnHole ซึ่งนอกจากตรวจไม่ให้ซ้ำกับตำแหน่งรถสิ่งกีดขวางแล้ว ยังตรวจสอบไม่ให้เกิดทับซ้อนกับเหรียญในระยะ 60 พิกเซล และไม่ทับซ้อนกับหลุมอื่นในพื้นที่ เพื่อให้ผู้เล่นสามารถมองเห็นและหลบหลีกสิ่งกีดขวางได้อย่างมีประสิทธิภาพ อยู่ที่ CarGUI.java

```
private void trySpawnHole() {
    Hole availableHole = getAvailableHole();
    if (availableHole == null) return;

    int[] lanesX = getLaneX();
    List<Integer> safeLanes = new ArrayList<>();

    for (int laneX : lanesX) {
        if (isSafeToSpawnHole(laneX)) {
            safeLanes.add(laneX);
        }
    }

    if (!safeLanes.isEmpty()) {
        int randomLaneX = safeLanes.get(random.nextInt(safeLanes.size()));
        int holeX = randomLaneX + (CAR_WIDTH - 110) / 2;
        availableHole.spawn(holeX, -110);
    }
}

private boolean isSafeToSpawnHole(int laneX) {
    int holeX = laneX + (CAR_WIDTH - 110) / 2;

    for (ObstacleCar obs : obstaclePool) {
        if (obs.isActive() && obs.getX() == laneX && obs.getY() >= -CAR_HEIGHT && obs.getY() <= 350) {
            return false;
        }
    }

    for (Coin c : coinPool) {
        if (c.isActive() && Math.abs(c.getX() - holeX) < 60 && c.getY() >= -120 && c.getY() <= 350) {
            return false;
        }
    }

    for (Hole h : holePool) {
        if (h.isActive() && h.getX() == holeX && h.getY() >= -110 && h.getY() <= 350) {
            return false;
        }
    }

    return true;
}
```

- Police Bottom-Spawning Algorithm

- อัลกอริทึมปล่อยรถตำรวจเคลื่อนที่จากด้านล่างขึ้นด้านบน (Reverse Movement Spawn)
เมื่อคะแนนถึงเงื่อนไข เมื่อ policeReadyToSpawn เป็นจริง ฟังก์ชัน trySpawnPolice จะ
ค้นหาเลนที่ไม่มีรถสิ่งกีดขวางใช้งานอยู่ผ่าน isPoliceLaneClear เมื่อพบเลนว่าง จะสั่งเรียก
policeCar.spawn(randomLane) ซึ่งส่งพิกัด Y ไปที่ 1080 (ขอบล่างสุดของหน้าจอ)
เพื่อให้รถตำรวจวิ่งสวนขึ้นมาจากด้านหลังผู้เล่น อยู่ที่ CarGUI.java และ PoliceCar.java

```
private void trySpawnPolice() {  
    int[] lanesX = getLaneX();  
    List<Integer> clearLanes = new ArrayList<>();  
  
    for (int laneX : lanesX) {  
        if (isPoliceLaneClear(laneX)) {  
            clearLanes.add(laneX);  
        }  
    }  
  
    if (!clearLanes.isEmpty()) {  
        int randomLane = clearLanes.get(random.nextInt(clearLanes.size()));  
        policeCar.spawn(randomLane);  
        policeReadyToSpawn = false;  
    }  
}
```

```
public void spawn(int startX) {  
    super.spawn(startX, 1080);  
}
```

- Axis-Aligned Bounding Box Intersection & Collision Explosion Centroid Algorithm

- อัลกอริทึมตรวจสอบการชนกันของเอนทิตีโดยใช้รูปสี่เหลี่ยมผืนผ้าขนานแกน (AABB) พร้อมคำนวณจุดศูนย์กลางบริเวณที่ชนกันเพื่อแสดงเอฟเฟกต์การระเบิด ในการอัปเดตตำแหน่งทุกๆ เฟรม อัลกอริทึมจะดึงขอบเขตสี่เหลี่ยมของตัวผู้เล่น (playerBounds) และขอบเขตของสิ่งกีดขวาง (obstacleBounds) มาทำการตรวจสอบการตัดกันด้วยเมธอด intersects หากมีการชนกันเกิดขึ้น อัลกอริทึมจะใช้เมธอด intersection เพื่อหาพื้นที่สี่เหลี่ยมเฉพาะส่วนที่ซ้อนทับกัน แล้วคำนวณจุดศูนย์กลาง (getCenterX(), getCenterY()) ของพื้นที่นั้น เพื่อนำพิกัดไปวางเอฟเฟกต์ภาพระเบิดควัน (smokeLabel) ให้ตรงกับจุดที่ปะทะกันอย่างถูกต้อง ก่อนเรียกคำสั่ง gameOver อยู่ที่ ObstacleCar.java, Hole.java, PoliceCar.java

```
@Override
public void updatePosition() {
    if (!active || gui.isGameOver()) return;

    int nextY = getY() + speed;
    setLocation(getX(), nextY);

    Rectangle playerBounds = gui.carPanel.getBounds();
    Rectangle obstacleBounds = getBounds();

    if (playerBounds.intersects(obstacleBounds)) {
        Rectangle intersection = playerBounds.intersection(obstacleBounds);
        int centerX = (int) intersection.getCenterX();
        int centerY = (int) intersection.getCenterY();
        gui.gameOver(centerX - 70, centerY - 66);
        return;
    }

    if (nextY > 1080) {
        reset();
    }
}
```

- Item Collection & Reward Algorithm

- อัลกอริทึมตรวจสอบการเก็บเหรียญเพื่อเพิ่มคะแนนโบนัสและคืนค่าเอนิติเหรียญกลับเข้าสู่พูล เหรียญ เคลื่อนที่เหรียญลงมาตามแกน Y ด้วยค่า speed และตรวจสอบการซ้อนทับขอบเขตกับรถผู้เล่นผ่าน intersects เมื่อเกิดการชนกัน อัลกอริทึมจะเพิ่มคะแนนสะสม 5,000 คะแนนให้ผู้เล่นทันทีผ่าน gui.addScore(5000) แล้วทำการ reset() เหรียญนั้นกลับเข้าสู่สถานะพร้อมใช้งานใหม่ทันที อยู่ที่ Coin.java

```
@Override
public void updatePosition() {
    if (!active || gui.isGameOver()) return;

    int nextY = getY() + speed;
    setLocation(getX(), nextY);

    Rectangle playerBounds = gui.carPanel.getBounds();
    Rectangle coinBounds = getBounds();

    if (playerBounds.intersects(coinBounds)) {
        gui.addScore(5000);
        reset();
        return;
    }

    if (nextY > 1080) {
        reset();
    }
}
```

- Finish Line Detection & Victory Condition Logic

- อัลกอริทึมการตรวจสอบเงื่อนไขความพร้อม การเลื่อนตำแหน่งเส้นชัย และการตรวจจับการเข้าเส้นชัยเพื่อชนะเกม อยู่ภายในเมธอด initMasterGameLoop เมื่อคะแนนสะสมครบ 100,000 คะแนนและพื้นที่ขอบบนสุดของถนนไม่มีรถสิ่งกีดขวางขยับ (isFinishLineAreaClear()) อัลกอริทึมจะเปิดการแสดงผลเส้นชัย และเลื่อนพิกัด finishLineY ลงมาเรื่อยๆ จากนั้นทำการตรวจจับการตัดกันระหว่างขอบเขตรถผู้เล่น (carBounds) กับขอบเขตเส้นชัย (finishBounds) เมื่ออินเตอร์เซกต์กัน ระบบจะเรียกเมธอด winGame() เพื่อหยุดเกมและแสดงหน้าจอชนะ อยู่ที่ CarGUI.java

```
if (readyForFinishLine && !finishLineSpawned) {  
    if (isFinishLineAreaClear()) {  
        finishLineSpawned = true;  
        finishLineLabel.setVisible(true);  
    }  
}  
  
if (finishLineSpawned) {  
    finishLineY += 3;  
    finishLineLabel.setLocation(finishLineX, finishLineY);  
  
    Rectangle carBounds = carPanel.getBounds();  
    Rectangle finishBounds = finishLineLabel.getBounds();  
    if (carBounds.intersects(finishBounds)) {  
        winGame();  
    }  
}
```

- Continuous Infinite Track Scrolling Algorithm

- อัลกอริทึมการเรนเดอร์ภาพพื้นหลังถนนแบบวนลูบไร้สิ้นสุด (Infinite Scrolling Background) โดยใช้เทคนิคการวาดภาพ 2 ภาพต่อกัน ในทุกๆ เฟรม เมธอด `updateScroll` จะเพิ่มค่า `trackY` ด้วย `scrollSpeed` (10 พิกเซล) แล้วใช้ตัวดำเนินการ `% 1080` เพื่อให้ค่าพิกัดวนลูบกลับมาอยู่ในช่วง 0 ถึง 1079 เสมอ ในส่วนเมธอด `paintComponent` จะทำการวาดภาพถนนสองใบต่อเนื่องกัน ใบแรกวาดที่พิกัด `trackY` และใบที่สองวาดต่อด้านบนที่พิกัด `trackY - 1080` ทำให้ผู้เล่นรู้สึกว่ารถกำลังวิ่งไปข้างหน้า บนถนนอย่างต่อเนื่องไม่สิ้นสุด อยู่ที่ `CarGUI.java`

```
public void updateScroll() {
    trackY = (trackY + scrollSpeed) % 1080;
    repaint();
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    if (trackImage != null) {
        g.drawImage(trackImage, 0, trackY, 800, 1080, null);
        g.drawImage(trackImage, 0, trackY - 1080, 800, 1080, null);
    }
}
```

- Custom Geometric Progress Gauge Algorithm

- อัลกอริทึมการวาดหลอดความก้าวหน้า ด้วยกราฟิกเวกเตอร์เชิงค่านวณ เมธอด `setScore` ทำการจำกัดขอบเขตค่าคะแนนให้อยู่ในช่วง 0 ถึง `maxScore` (100,000) ในเมธอด `paintComponent` จะคำนวณสัดส่วนความก้าวหน้า $progress = currentScore / maxScore$ เพื่อแปลงเป็นความกว้างพิกเซล `fillWidth` จากนั้นใช้ออบเจกต์ `Graphics2D` วาดพื้นหลังหลอด วาดแถบเติมสีตามความก้าวหน้า และวาดวงกลมสัญลักษณ์นำทางบริเวณปลายแถบสะสมความก้าวหน้า อยู่ที่ `CarGUI.java`

```
public void setScore(int score) {
    this.currentScore = Math.min(Math.max(score, 0), maxScore);
    repaint();
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g.create();
    try {
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        int barX = 20;
        int barY = 20;
        int barWidth = 320;
        int barHeight = 18;
        int circleRadius = 15;

        g2d.setColor(Color.BLACK);
        g2d.fillRect(barX, barY, barWidth, barHeight);

        double progress = (double) currentScore / maxScore;
        int fillWidth = (int) (progress * barWidth);

        g2d.setColor(new Color(254, 224, 126));
        g2d.fillRect(barX, barY, fillWidth, barHeight);

        g2d.setColor(new Color(40, 40, 40));
        g2d.setStroke(new BasicStroke(2));
        g2d.drawRect(barX, barY, barWidth, barHeight);

        int circleX = barX + fillWidth;
        int circleY = barY + (barHeight / 2);

        g2d.setColor(new Color(255, 226, 128));
        g2d.fillOval(circleX - circleRadius, circleY - circleRadius,
            circleRadius * 2, circleRadius * 2);

        g2d.setColor(Color.BLACK);
        g2d.setStroke(new BasicStroke(2.5f));
        g2d.drawOval(circleX - circleRadius, circleY - circleRadius,
            circleRadius * 2, circleRadius * 2);
    } finally {
        g2d.dispose();
    }
}
```

- Font & Resource Registration Engine Algorithm

- อัลกอริทึมการโหลดและลงทะเบียนฟอนต์ภายนอกเข้าสู่ระบบพร้อมระบบจัดการ

ข้อผิดพลาด ฟังก์ชันทำหน้าที่อ่านไฟล์ฟอนต์ TrueType (.ttf) จากดิสก์และสร้างออบเจกต์

Font กำหนดขนาดและรูปแบบตามระบุ ในกรณีที่ไม่สามารถโหลดฟอนต์ดังกล่าวได้

อัลกอริทึมจะมีระบบสำรองข้อมูล โดยจะเปลี่ยนไปสร้างฟอนต์มาตรฐานของระบบ เช่น

"Impact" หรือ "Serif" เพื่อป้องกันไม่ให้โปรแกรมหยุดทำงาน อยู่ที่ TitleGUI.java,

LevelSelectGUI.java, CarGUI.java

```
private Font loadCustomFont(float size) {  
    try {  
        File fontFile = new File("asset/SedgwickAveDisplay-Regular.ttf");  
        return Font.createFont(Font.TRUETYPE_FONT, fontFile).deriveFont(Font.BOLD, size);  
    } catch (Exception e) {  
        return new Font("Impact", Font.BOLD, (int) size);  
    }  
}
```

บทที่ 3 สรุป

ปัญหาที่พบระหว่างการพัฒนา

1. เมื่อนำ ไฟล์ Gif มารันเป็นสนาม ส่งผลให้ Timer ทำงานหนักจนสนามกระพริบหายเป็นพัก ๆ
2. ความยากที่มากเกินไป และการเกิดสุมในบางครั้งก็บังทางของผู้เล่นมากเกินไป
3. การ Spawn ของที่เยอะเกินไป และ ต้องคอยลบสร้างใหม่ซ้ำๆ

จุดเด่นของโปรแกรม

1. ระบบการเกิดสิ่งกีดขวาง

โค้ดไม่ได้สุมเกิดสิ่งกีดขวางแบบสุมสี่สุมห้า แต่มียุทธวิธีตรวจสอบความปลอดภัยก่อนปล่อยวัตถุ (isSafeToSpawn) เพื่อรับประกันว่าผู้เล่นจะมีช่องทางรอดเสมอ ตรวจสอบระยะห่างแนวตั้งขั้นต่ำ (320 พิกเซล), จำกัดจำนวนเลนที่ถูกบล็อกพร้อมกันไม่ให้เกิน 2 เลน และตรวจจับพิกัดตัวผู้เล่นเพื่อป้องกันการปล่อยสิ่งกีดขวางดักทางในมุมอับขอบถนน (Corner Trap)

2. การบริหารจัดการหน่วยความจำด้วย Object Pooling Pattern

คลาสฐาน GameObject ร่วมกับกลุ่มพูล (obstaclePool, coinPool, holePool) ถูกออกแบบมาเพื่อนำวัตถุที่พ้นขอบจอแล้วกลับมาซ่อนตำแหน่งไว้ที่ (-500, -500) แล้วนำกลับมาเปิดใช้งานใหม่ (spawn) หลีกเลียงคำสั่ง new เพื่อสร้างวัตถุใหม่ตลอดเวลา ช่วยลดปัญหาภาพกระตุก